ПРАКТИЧЕСКАЯ РАБОТА № 8

Анализ данных с помощью SQL-запросов

Выполнил: Лукашин Елисей Андреевич
Группа: ИСП-211
Проверил: _____

1. Цель работы

Освоить построение запросов к реляционной базе данных: соединение таблиц, агрегацию и группировку данных, подзапросы, представления, индексы, триггеры, транзакции и разграничение прав доступа.

2. Исходные данные

Запросы выполняются к базе данных lukashin_bd, созданной в практической работе № 7. База содержит 4 учебные группы, 8 студентов, 4 преподавателей, 5 дисциплины, 10 занятий и 15 оценок. Схема включает таблицы groups, students, teachers, disciplines, lessons и grades, связанные внешними ключами.

3. Запросы и результаты

3.1. Список всех студентов с названиями их групп

```
SELECT s.last_name, s.first_name, g.group_name
FROM students AS s
INNER JOIN `groups` AS g ON g.group_id = s.group_id
ORDER BY g.group_name, s.last_name;
```

INNER JOIN связывает студента с его группой по внешнему ключу group_id.

last_name	first_name	group_name
Антонов	Кирилл	ИСП-211
Богданова	Мария	ИСП-211
Власов	Егор	ИСП-211
Гордеева	Софья	ИСП-211
Лукашин	Елисей	ИСП-211
Данилов	Илья	ИСП-212
Егорова	Дарья	ИСП-212
Капустин	Роман	СА-211

Получено строк: 8.

3.2. Количество студентов в каждой группе

```
SELECT g.group_name, COUNT(s.student_id) AS students_count
FROM `groups` AS g
LEFT JOIN students AS s ON s.group_id = g.group_id
GROUP BY g.group_id, g.group_name
ORDER BY students_count DESC, g.group_name;
```

LEFT JOIN сохраняет в результате группы, в которых ещё нет студентов.

group_name	students_count
ИСП-211	5
ИСП-212	2
СА-211	1
СА-212	0

Получено строк: 4.

3.3. Расписание занятий с полной информацией

```
SELECT l.lesson_date, g.group_name, d.discipline_name,
       t.last_name, t.first_name, l.lesson_type
FROM lessons AS l
INNER JOIN `groups` AS g     ON g.group_id = l.group_id
INNER JOIN disciplines AS d  ON d.discipline_id = l.discipline_id
INNER JOIN teachers AS t     ON t.teacher_id = l.teacher_id
ORDER BY l.lesson_date, g.group_name;
```

Четыре таблицы объединены последовательными INNER JOIN.

lesson_date	group_name	discipline_name	last_name	first_name	lesson_type
2025-09-01	ИСП-211	Информатика	Волков	Дмитрий	Лекция
2025-09-03	ИСП-211	Информатика	Волков	Дмитрий	Практика
2025-09-05	ИСП-211	Алгоритмы и структуры данных	Панова	Елена	Лекция
2025-09-08	ИСП-212	Алгоритмы и структуры данных	Панова	Елена	Практика
2025-09-10	ИСП-212	Проектирование баз данных	Романов	Артём	Лекция
2025-09-12	СА-211	Проектирование баз данных	Романов	Артём	Практика
2025-09-15	СА-211	Объектно-ориентированное программирование	Семёнова	Наталья	Лекция
2025-09-17	ИСП-211	Объектно-ориентированное программирование	Семёнова	Наталья	Лабораторная
2025-09-19	ИСП-212	Компьютерные сети	Волков	Дмитрий	Лекция
2025-09-22	СА-211	Компьютерные сети	Панова	Елена	Практика

Получено строк: 10.

3.4. Нагрузка преподавателей (количество занятий)

```
SELECT t.last_name, t.first_name, COUNT(l.lesson_id) AS lessons_count
FROM teachers AS t
LEFT JOIN lessons AS l ON l.teacher_id = t.teacher_id
GROUP BY t.teacher_id, t.last_name, t.first_name
ORDER BY lessons_count DESC;
```

LEFT JOIN показывает и преподавателей без занятий (0).

last_name	first_name	lessons_count
Волков	Дмитрий	3
Панова	Елена	3
Романов	Артём	2
Семёнова	Наталья	2

Получено строк: 4.

3.5. Студенты, родившиеся после 2005 года

```
SELECT last_name, first_name, birth_date
FROM students
WHERE birth_date > '2005-12-31'
ORDER BY birth_date;
```

Сравнение дат: условие отбирает студентов 2006 года рождения и моложе.

last_name	first_name	birth_date
Богданова	Мария	2006-02-27
Гордеева	Софья	2006-03-30
Егорова	Дарья	2006-05-04

Получено строк: 3.

3.6. Дисциплины с количеством часов больше порога

```
SELECT discipline_name, hours
FROM disciplines
WHERE hours > 100
ORDER BY hours DESC;
```

Простой фильтр WHERE по числовому полю.

discipline_name	hours
Объектно-ориентированное программирование	132
Алгоритмы и структуры данных	120
Информатика	102

Получено строк: 3.

3.7. Группы, в которых больше одного студента

```
SELECT g.group_name, COUNT(s.student_id) AS students_count
FROM `groups` AS g
INNER JOIN students AS s ON s.group_id = g.group_id
GROUP BY g.group_id, g.group_name
HAVING COUNT(s.student_id) > 1
ORDER BY students_count DESC;
```

HAVING фильтрует уже сгруппированные строки, WHERE — строки до группировки.

group_name	students_count
ИСП-211	5
ИСП-212	2

Получено строк: 2.

3.8. Средняя нагрузка на преподавателя

```
SELECT ROUND(AVG(cnt), 2) AS avg_lessons
FROM (SELECT teacher_id, COUNT(*) AS cnt
      FROM lessons
      GROUP BY teacher_id) AS t;
```

Подзапрос в FROM возвращает количество занятий по каждому преподавателю.

avg_lessons
2.5

Получено строк: 1.

3.9. Список специальностей групп (без дубликатов)

```
SELECT DISTINCT specialty
FROM `groups`
ORDER BY specialty;
```

DISTINCT убирает повторяющиеся значения.

specialty
Информационные системы и программирование
Системное администрирование

Получено строк: 2.

3.10. Студенты и их группы (с использованием подзапроса)

```
SELECT s.last_name, s.first_name,
       (SELECT g.group_name FROM `groups` AS g
        WHERE g.group_id = s.group_id) AS group_name
FROM students AS s
ORDER BY s.last_name;
```

Скалярный подзапрос в списке выборки возвращает название группы.

last_name	first_name	group_name
Антонов	Кирилл	ИСП-211
Богданова	Мария	ИСП-211
Власов	Егор	ИСП-211
Гордеева	Софья	ИСП-211
Данилов	Илья	ИСП-212
Егорова	Дарья	ИСП-212
Капустин	Роман	СА-211
Лукашин	Елисей	ИСП-211

Получено строк: 8.

3.11. Группы, в которых нет ни одного студента

```
SELECT g.group_name
FROM `groups` AS g
WHERE NOT EXISTS (SELECT 1 FROM students AS s
                  WHERE s.group_id = g.group_id)
ORDER BY g.group_name;
```

Коррелированный подзапрос с NOT EXISTS.

group_name
СА-212

Получено строк: 1.

3.12. Средний балл каждого студента

```
SELECT s.last_name, s.first_name,
       COUNT(gr.grade_id) AS grades_count,
       ROUND(AVG(gr.grade), 2) AS avg_grade
FROM students AS s
INNER JOIN grades AS gr ON gr.student_id = s.student_id
GROUP BY s.student_id, s.last_name, s.first_name
ORDER BY avg_grade DESC, s.last_name;
```

Агрегация оценок по студенту.

last_name	first_name	grades_count	avg_grade
Богданова	Мария	3	5.0

last_name	first_name	grades_count	avg_grade
Данилов	Илья	1	5.0
Лукашин	Елисей	3	4.67
Власов	Егор	2	4.0
Гордеева	Софья	1	4.0
Капустин	Роман	1	4.0
Антонов	Кирилл	3	3.67
Егорова	Дарья	1	3.0

Получено строк: 8.

3.13. Средний балл по дисциплинам

```
SELECT d.discipline_name,
       COUNT(gr.grade_id) AS grades_count,
       ROUND(AVG(gr.grade), 2) AS avg_grade
FROM disciplines AS d
INNER JOIN grades AS gr ON gr.discipline_id = d.discipline_id
GROUP BY d.discipline_id, d.discipline_name
ORDER BY avg_grade DESC;
```

Агрегация оценок по дисциплине.

discipline_name	grades_count	avg_grade
Компьютерные сети	1	5.0
Алгоритмы и структуры данных	4	4.5
Объектно-ориентированное программирование	3	4.33
Информатика	4	4.0
Проектирование баз данных	3	4.0

Получено строк: 5.

3.14. Лучшие студенты (средний балл выше 4,5)

```
SELECT s.last_name, s.first_name, ROUND(AVG(gr.grade), 2) AS avg_grade
FROM students AS s
INNER JOIN grades AS gr ON gr.student_id = s.student_id
GROUP BY s.student_id, s.last_name, s.first_name
HAVING AVG(gr.grade) > 4.5
ORDER BY avg_grade DESC;
```

Самостоятельное задание: HAVING с агрегатной функцией.

last_name	first_name	avg_grade
Богданова	Мария	5.0
Данилов	Илья	5.0
Лукашин	Елисей	4.67

Получено строк: 3.

3.15. Распределение студентов по уровню успеваемости

```
SELECT s.last_name, s.first_name, ROUND(AVG(gr.grade), 2) AS avg_grade,
       CASE
           WHEN AVG(gr.grade) >= 4.5 THEN 'отлично'
           WHEN AVG(gr.grade) >= 3.5 THEN 'хорошо'
           ELSE 'удовлетворительно'
       END AS level
FROM students AS s
INNER JOIN grades AS gr ON gr.student_id = s.student_id
GROUP BY s.student_id, s.last_name, s.first_name
ORDER BY avg_grade DESC;
```

Выражение CASE формирует текстовую категорию по среднему баллу.

last_name	first_name	avg_grade	level
Богданова	Мария	5.0	отлично
Данилов	Илья	5.0	отлично
Лукашин	Елисей	4.67	отлично
Власов	Егор	4.0	хорошо
Гордеева	Софья	4.0	хорошо
Капустин	Роман	4.0	хорошо
Антонов	Кирилл	3.67	хорошо
Егорова	Дарья	3.0	удовлетворительно

Получено строк: 8.

4. Работа с транзакциями (TCL)

Транзакция объединяет несколько операций в единое целое: если хотя бы одна из них завершится ошибкой, все изменения откатываются командой ROLLBACK. Это основа надёжности базы данных.

```
START TRANSACTION;

INSERT INTO grades (student_id, discipline_id, grade_date, grade)
VALUES (1, 1, '2025-10-01', 5),
       (2, 1, '2025-10-01', 4);

-- проверка внутри транзакции
SELECT student_id, grade FROM grades WHERE grade_date = '2025-10-01';

-- фиксация изменений
COMMIT;

-- пример отката
START TRANSACTION;
DELETE FROM grades WHERE grade_date = '2025-10-01';
ROLLBACK;
SELECT COUNT(*) FROM grades WHERE grade_date = '2025-10-01';  -- записи на месте
```

После COMMIT изменения сохраняются в базе, после ROLLBACK — отменяются полностью. В MySQL для незавершённой транзакции также работает неявный откат при разрыве соединения.

5. Создание представлений (VIEW)

Представление — виртуальная таблица, которая не хранит данные, а формируется из запроса при каждом обращении. Оно упрощает сложные запросы и позволяет разграничить доступ: пользователю можно выдать права только на представление, а не на исходные таблицы.

```
CREATE OR REPLACE VIEW v_student_groups AS
SELECT s.student_id,
       s.last_name,
       s.first_name,
       g.group_name,
       g.specialty
FROM students AS s
INNER JOIN `groups` AS g ON g.group_id = s.group_id;

SELECT * FROM v_student_groups ORDER BY group_name, last_name;
```

student_id	last_name	first_name	group_name	specialty
2	Антонов	Кирилл	ИСП-211	Информационные системы и программирование
3	Богданова	Мария	ИСП-211	Информационные системы и программирование
4	Власов	Егор	ИСП-211	Информационные системы и программирование
5	Гордеева	Софья	ИСП-211	Информационные системы и программирование
1	Лукашин	Елисей	ИСП-211	Информационные системы и программирование
6	Данилов	Илья	ИСП-212	Информационные системы и программирование
7	Егорова	Дарья	ИСП-212	Информационные системы и программирование
8	Капустин	Роман	СА-211	Системное администрирование

Получено строк: 8.

6. Создание индексов

Индекс ускоряет поиск и соединение таблиц, но замедляет вставку и обновление, поэтому создаётся по полям, которые часто используются в условиях WHERE и JOIN. Полезно проверить план выполнения запроса командой EXPLAIN: до создания индекса используется полный просмотр таблицы (type = ALL), после — поиск по индексу (type = ref).

```
CREATE INDEX idx_students_group   ON students (group_id);
CREATE INDEX idx_lessons_date     ON lessons (lesson_date);
CREATE INDEX idx_lessons_teacher  ON lessons (teacher_id);
CREATE INDEX idx_grades_student   ON grades (student_id);

EXPLAIN SELECT * FROM students WHERE group_id = 1;
```

Внешние ключи в InnoDB автоматически создают индексы по ссылающимся полям, поэтому дополнительные индексы по group_id и teacher_id нужны только при интенсивной аналитической нагрузке.

7. Триггеры

Триггер — процедура, автоматически выполняемая при вставке, обновлении или удалении строки. Ниже приведены два триггера: проверка диапазона оценки и журналирование изменений.

```
DELIMITER $$

-- 1. Запрет оценки вне диапазона 2…5
CREATE TRIGGER trg_grades_check
BEFORE INSERT ON grades
FOR EACH ROW
BEGIN
    IF NEW.grade < 2 OR NEW.grade > 5 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Оценка должна быть в диапазоне от 2 до 5';
    END IF;
END$$

-- 2. Журнал изменений оценок
CREATE TABLE grades_log (
    log_id     INT AUTO_INCREMENT PRIMARY KEY,
    grade_id   INT,
    old_grade  TINYINT,
    new_grade  TINYINT,
    changed_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER trg_grades_audit
AFTER UPDATE ON grades
FOR EACH ROW
BEGIN
    INSERT INTO grades_log (grade_id, old_grade, new_grade)
    VALUES (OLD.grade_id, OLD.grade, NEW.grade);
END$$

DELIMITER ;
```

Триггер trg_grades_check не позволит вставить некорректную оценку, а trg_grades_audit сохранит историю изменений: после выполнения команды UPDATE в таблице grades_log появится запись со старым и новым значением.

8. Разграничение прав доступа (DCL)

В реальной системе администратор видит всё, преподаватель — только студентов и оценки, студент — только свои результаты. Права выдаются командами GRANT и отзываются командой REVOKE.

```
CREATE ROLE 'teacher_role';
CREATE ROLE 'student_role';

CREATE USER 'teacher1'@'localhost' IDENTIFIED BY 'Teacher#2025';
CREATE USER 'student1'@'localhost' IDENTIFIED BY 'Student#2025';

GRANT SELECT ON `lukashin_bd`.v_student_groups TO 'teacher_role';
GRANT SELECT, INSERT, UPDATE ON `lukashin_bd`.grades TO 'teacher_role';

GRANT SELECT ON `lukashin_bd`.v_student_groups TO 'student_role';

GRANT 'teacher_role' TO 'teacher1'@'localhost';
GRANT 'student_role'  TO 'student1'@'localhost';
SET DEFAULT ROLE ALL TO 'teacher1'@'localhost', 'student1'@'localhost';

SHOW GRANTS FOR 'teacher1'@'localhost';

-- отзыв права
REVOKE INSERT ON `lukashin_bd`.grades FROM 'teacher_role';
```

Роль teacher_role получает чтение представления и изменение оценок, роль student_role — только чтение представления. Прямой доступ к таблицам студенту не выдаётся, поэтому он не увидит оценки других студентов.

9. Вывод

В работе построены запросы всех основных видов: соединения INNER и LEFT JOIN, агрегация с GROUP BY и HAVING, подзапросы в списке выборки и в условии, представления, индексы. Освоены транзакции, триггеры и управление правами доступа. Полученные результаты подтверждают корректность схемы базы данных, созданной в практической работе № 7.